

Deep Learning



William
of
Ockham

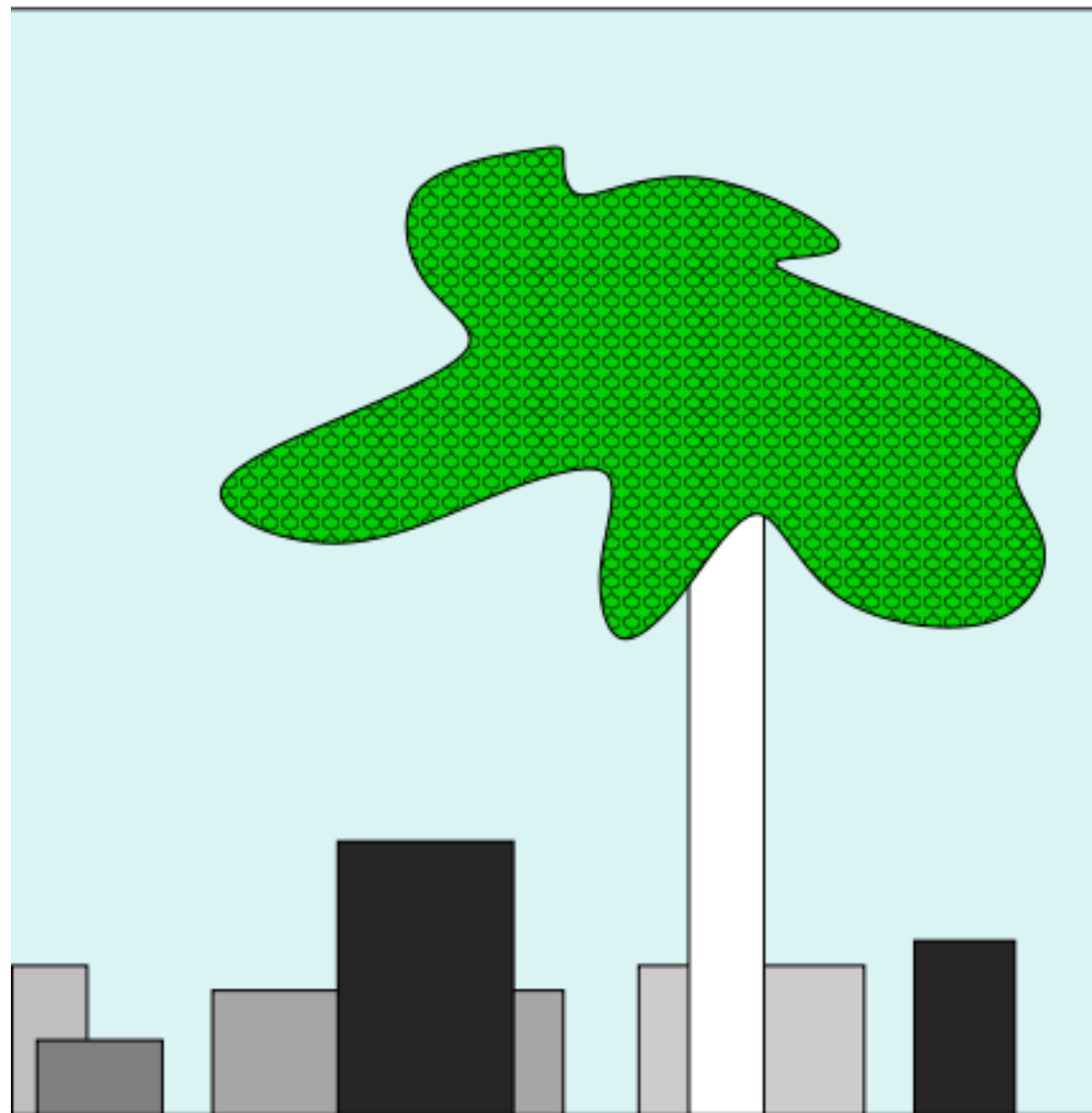
LECTURE 5B – OPTIMIZATION, GENERALIZATION, REGULARIZATION

Kevin Bryson

*Acknowledgements: Elements of Statistical Learning Theory
by Hastie, Tibshirani, Friedman.*

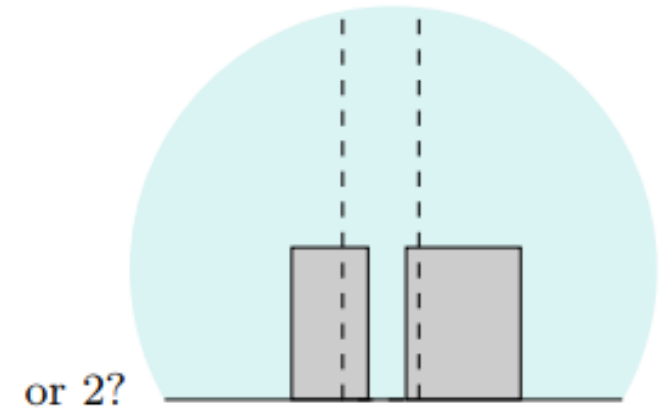
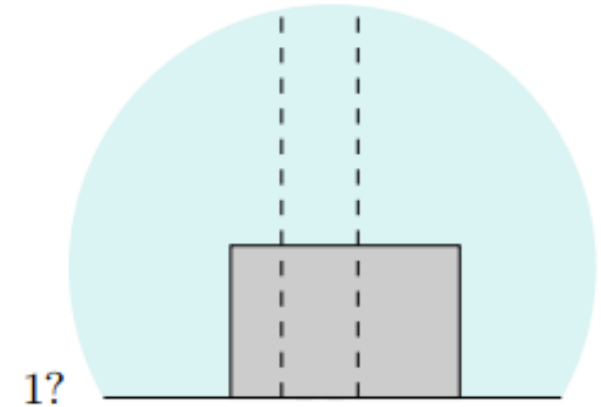
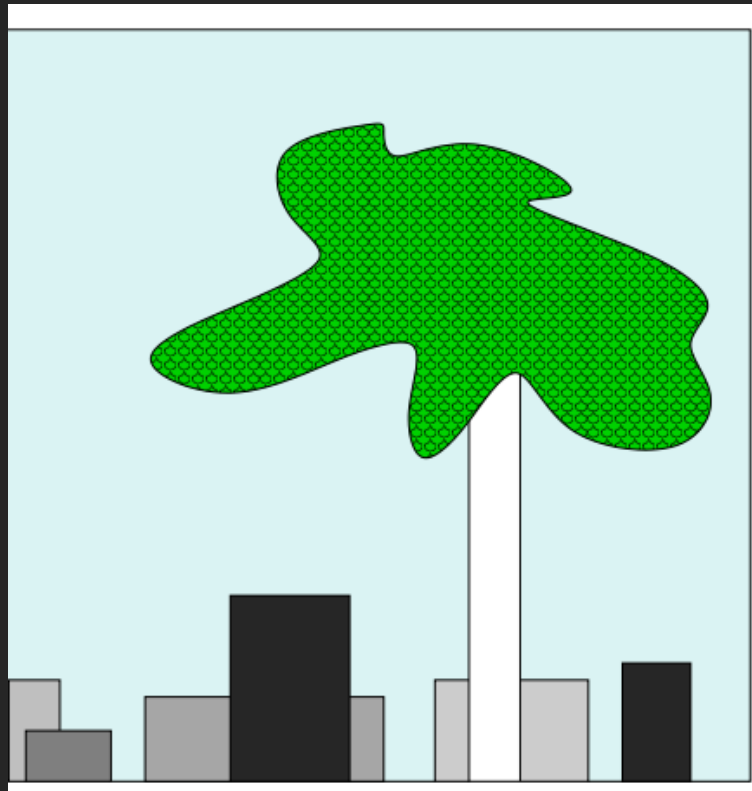
How many boxes are there?

(Mackay 2003)



How many boxes are there?

(Mackay 2003)



Ockham's razor

"Entities are not to be multiplied without necessity" (Non sunt multiplicanda entia sine necessitate)

Essentially states that simplest model that fits the data is more likely to be correct than complex ones.

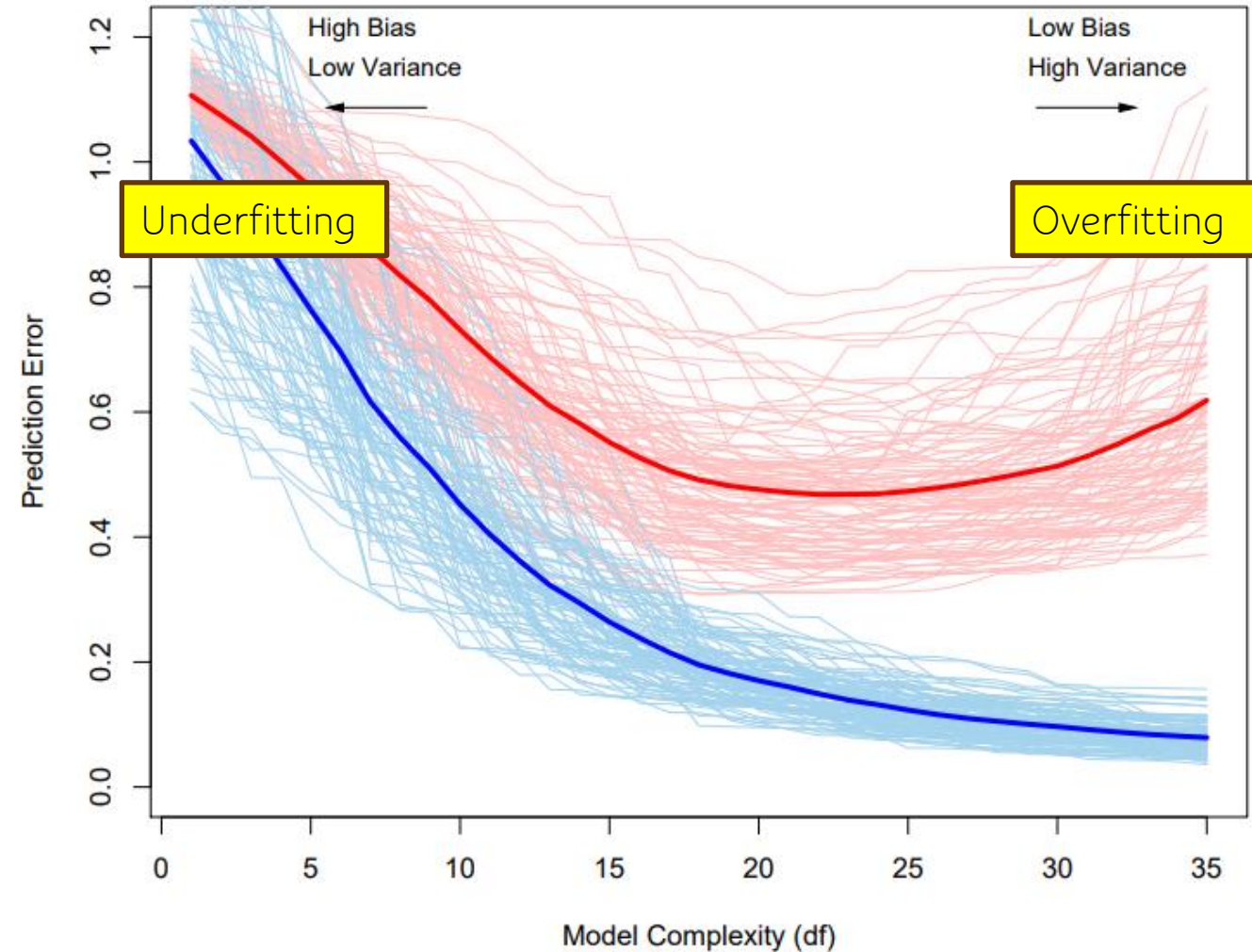


Bias, Variance and Model Complexity

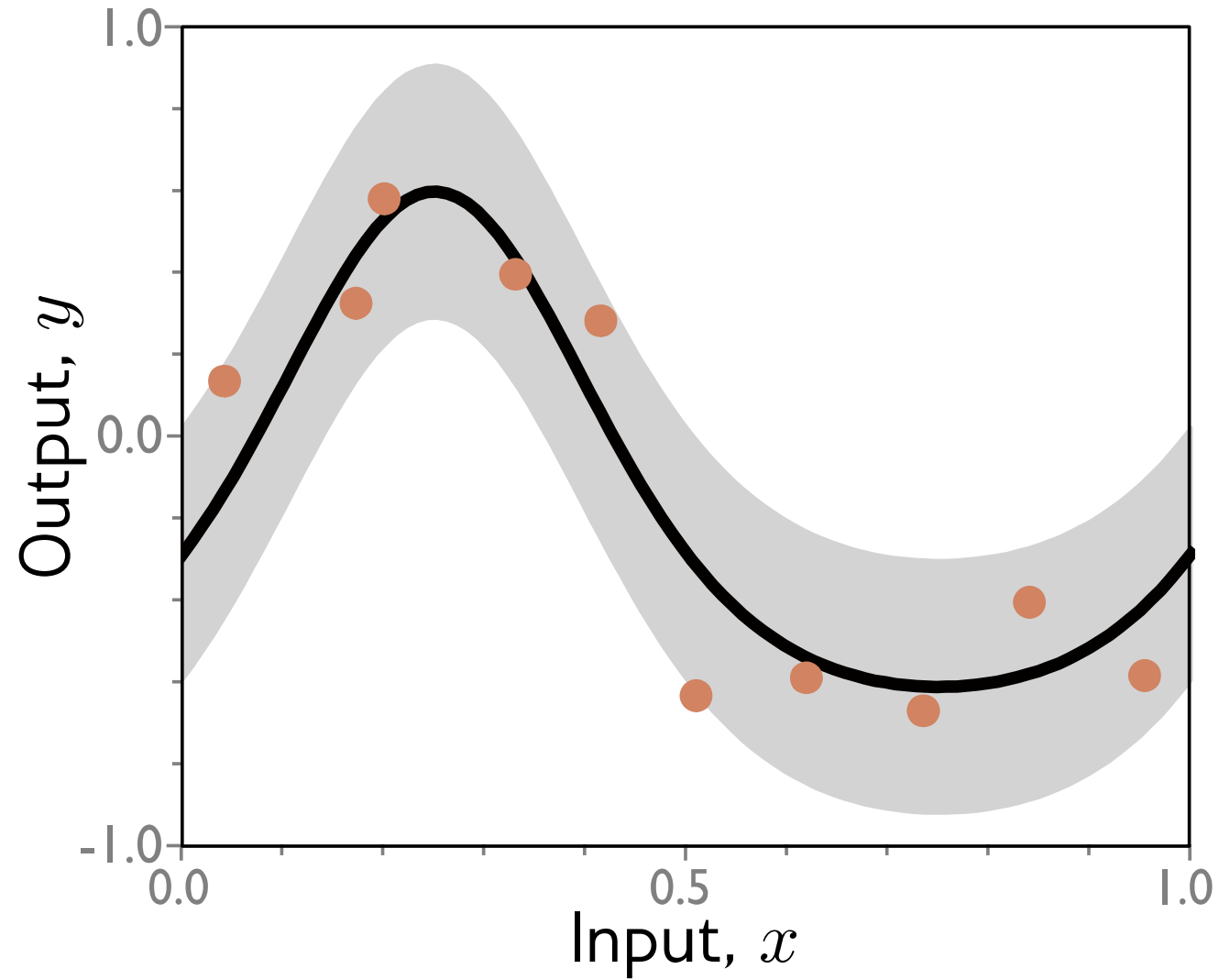
Curves in Red = different *test* sets

Curves in Blue = different *training* sets

1. Accuracy is a random variable depending on test & training sets (and many other things).
2. Good training set performance does not guarantee test set performance ! (Generalization ...)



Regression example



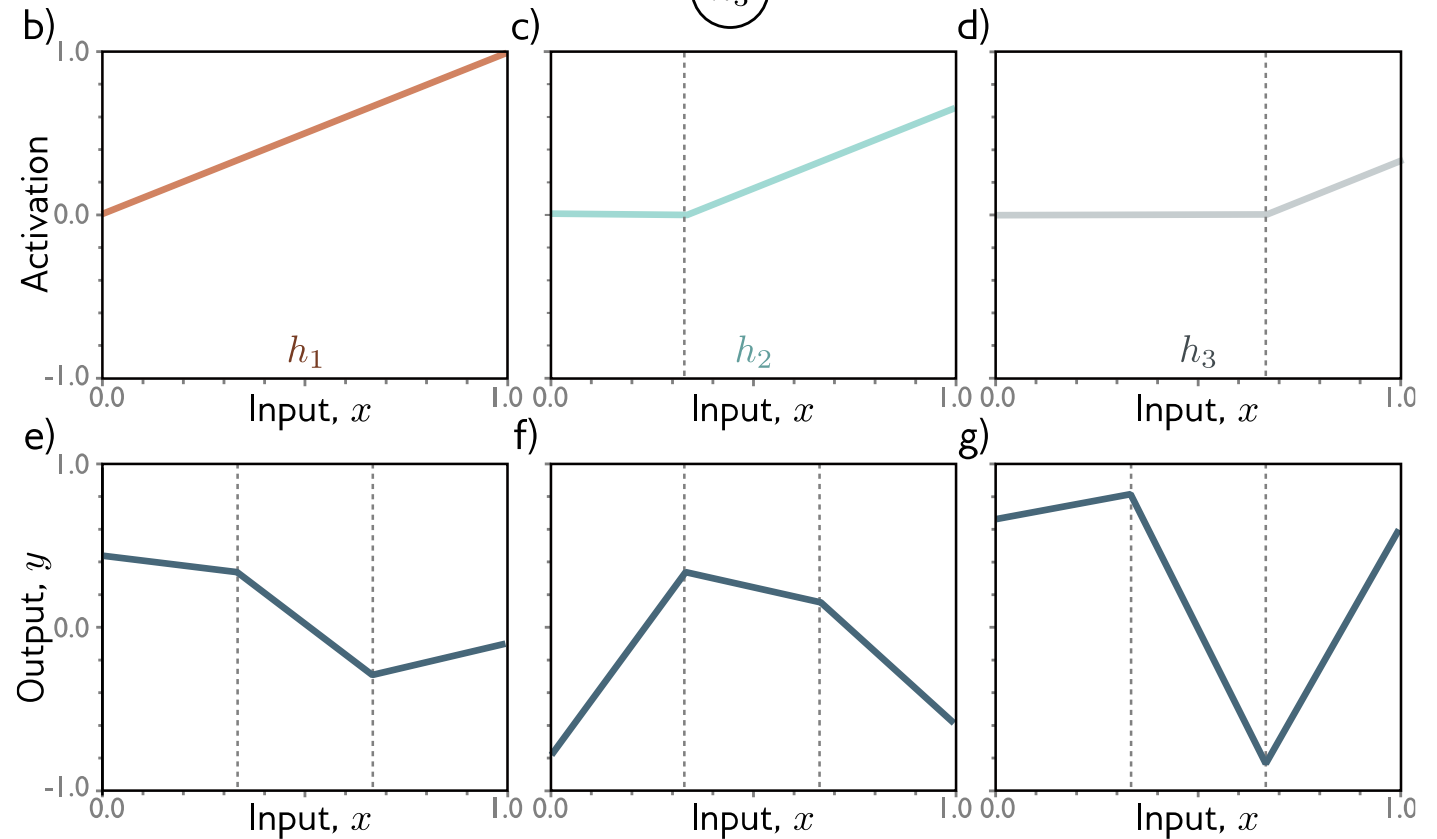
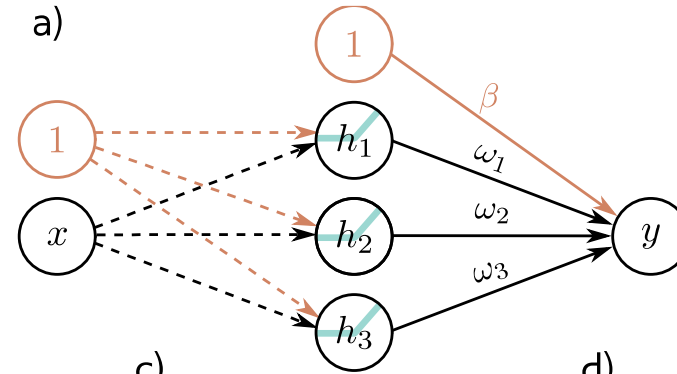
Toy neural network model

It has K hidden units.

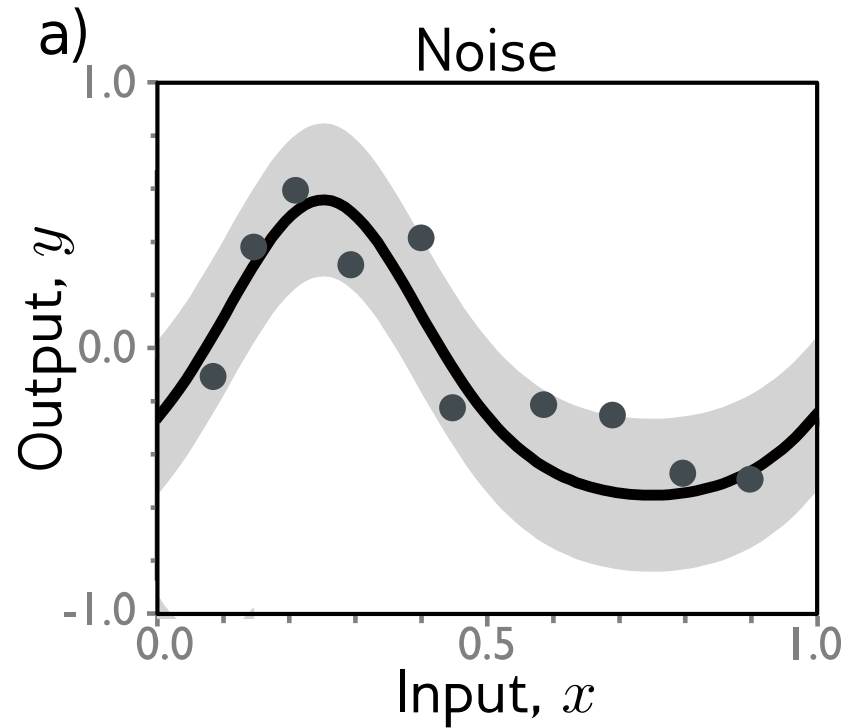
First layer fixed so “joints” divide interval evenly (this is what ReLUs do).

The second layer is trained to produce different piecewise linear functions.

But this model is then linear in h so convex loss function so can find best solutions in closed form (which means we can analyze it effectively).



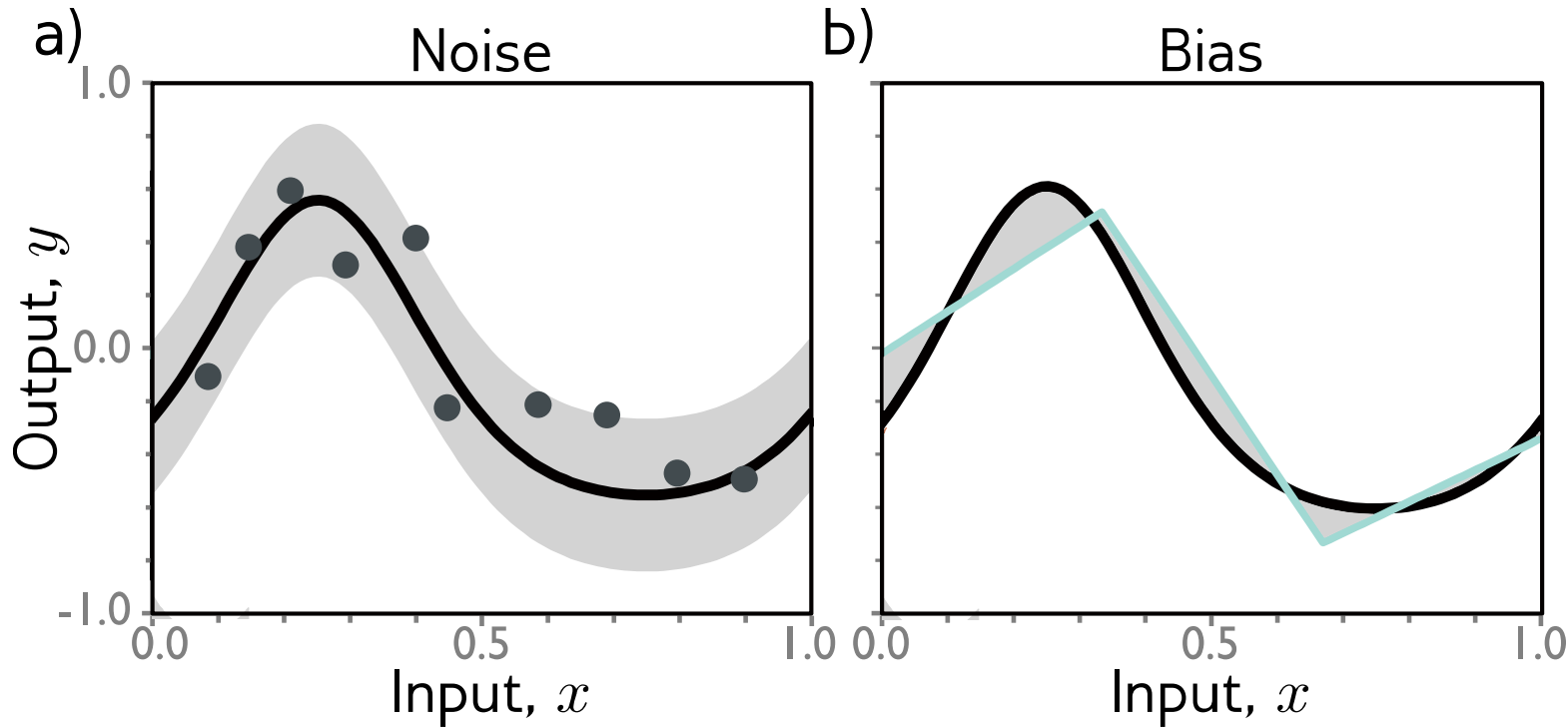
Noise, bias, and variance



- Noise in measurements
- Some variables not observed
- Data mislabeled

This is Bayes Error limits and cannot be avoided.

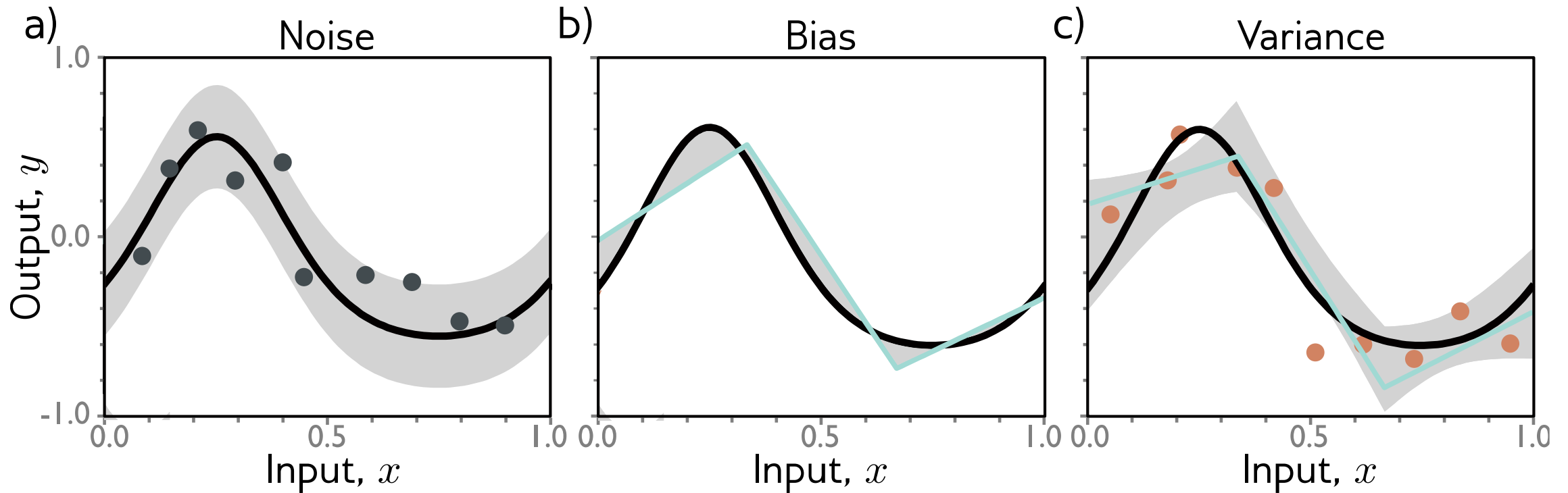
Noise, **bias**, and variance



- Bias is caused by the limitations of how well the model fits (i.e. the functions it can represent: 3-piece linear functions in this case).

Noise, bias, and variance

- Variance is how much the model varies according to training sets



Noise, bias, and variance

- ❑ Variance is the uncertainty in fitted model due to choice of training set
- ❑ Bias is systematic deviation from the mean of the function we are modeling due to limitations in our model
- ❑ Noise is inherent uncertainty in the true mapping from input to output

Least squares regression only

$$L[x] = (f[x, \phi] - y[x])^2$$

We can show that:

$$\mathbb{E}_{\mathcal{D}} \left[\mathbb{E}_y [L[x]] \right] = \underbrace{\mathbb{E}_{\mathcal{D}} \left[(f[x, \phi[\mathcal{D}]] - f_{\mu}[x])^2 \right]}_{\text{variance}} + \underbrace{(f_{\mu}[x] - \mu[x])^2}_{\text{bias}} + \underbrace{\sigma^2}_{\text{noise}}$$

Expectation over noise
in training data

Expectation over
noise in test data

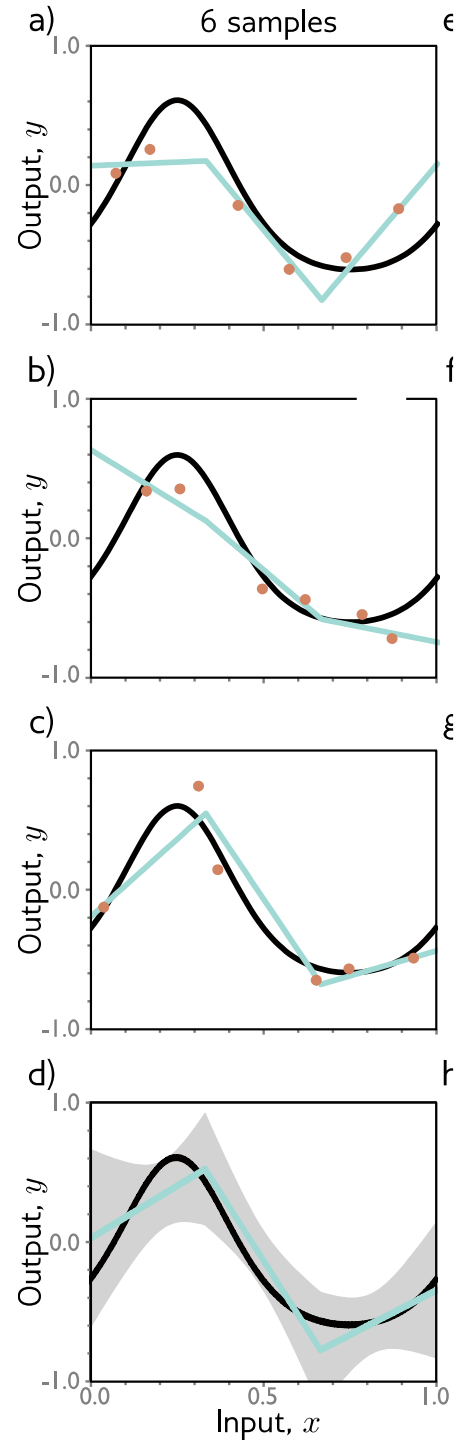
Actual model

Best possible model if
we had infinite data

True function

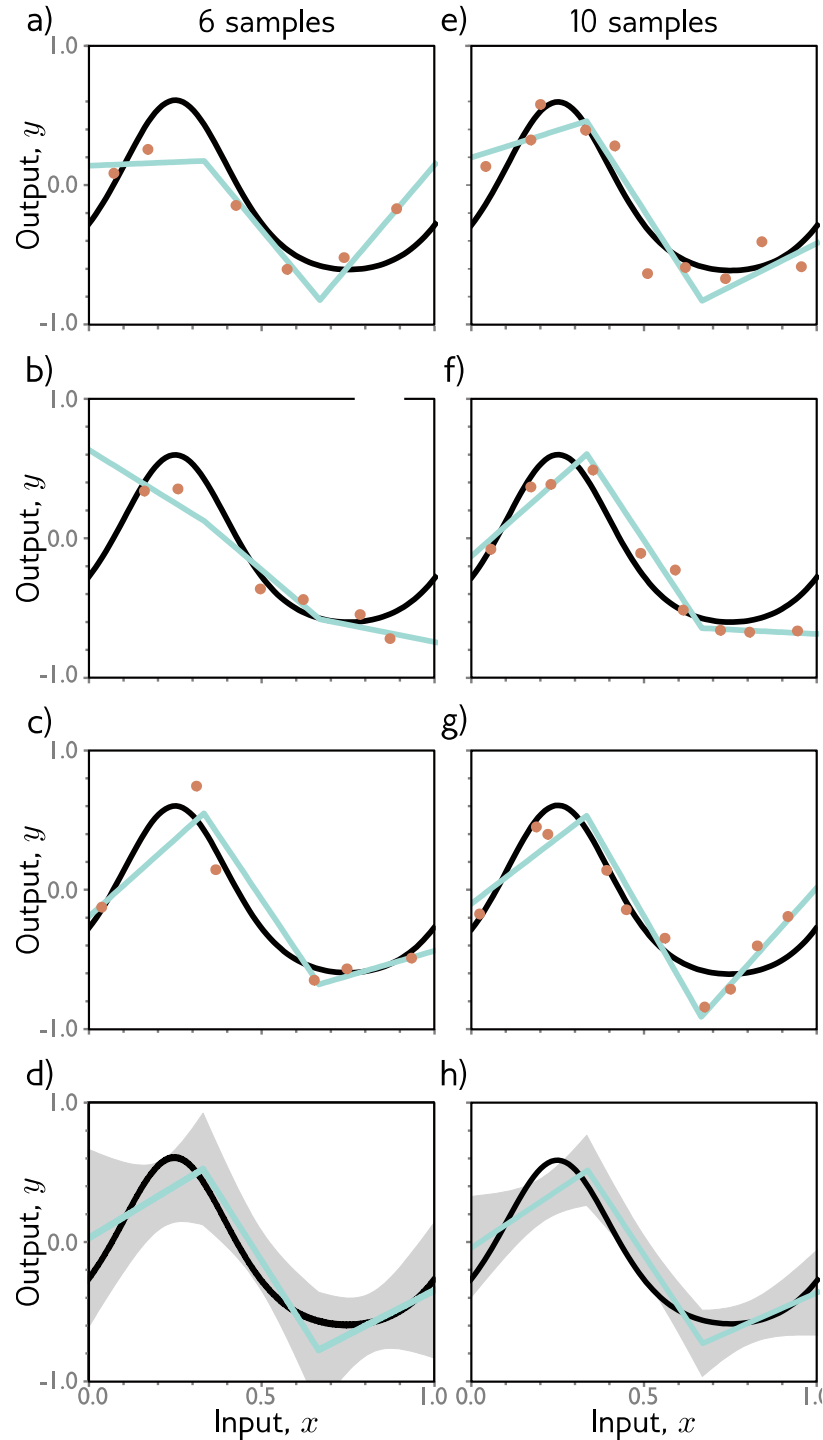
Reducing Variance:

More training data



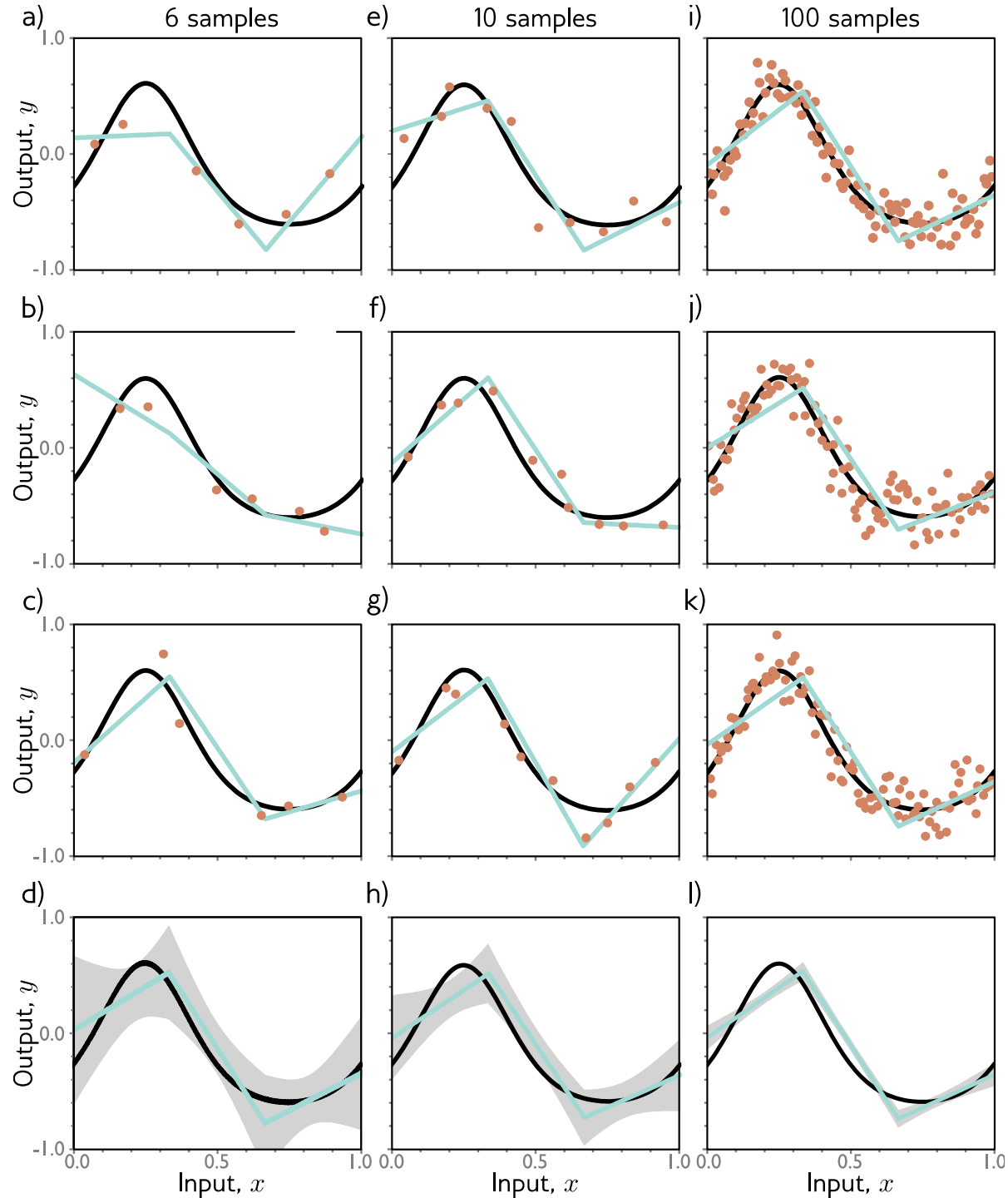
Reducing Variance:

More training data

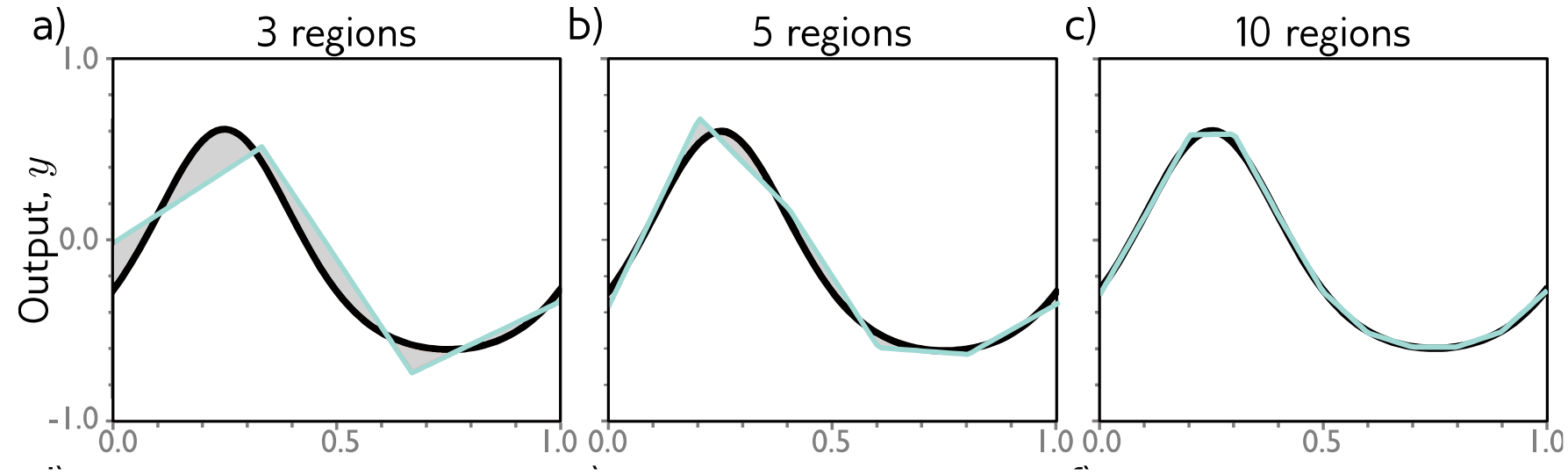


Reducing Variance:

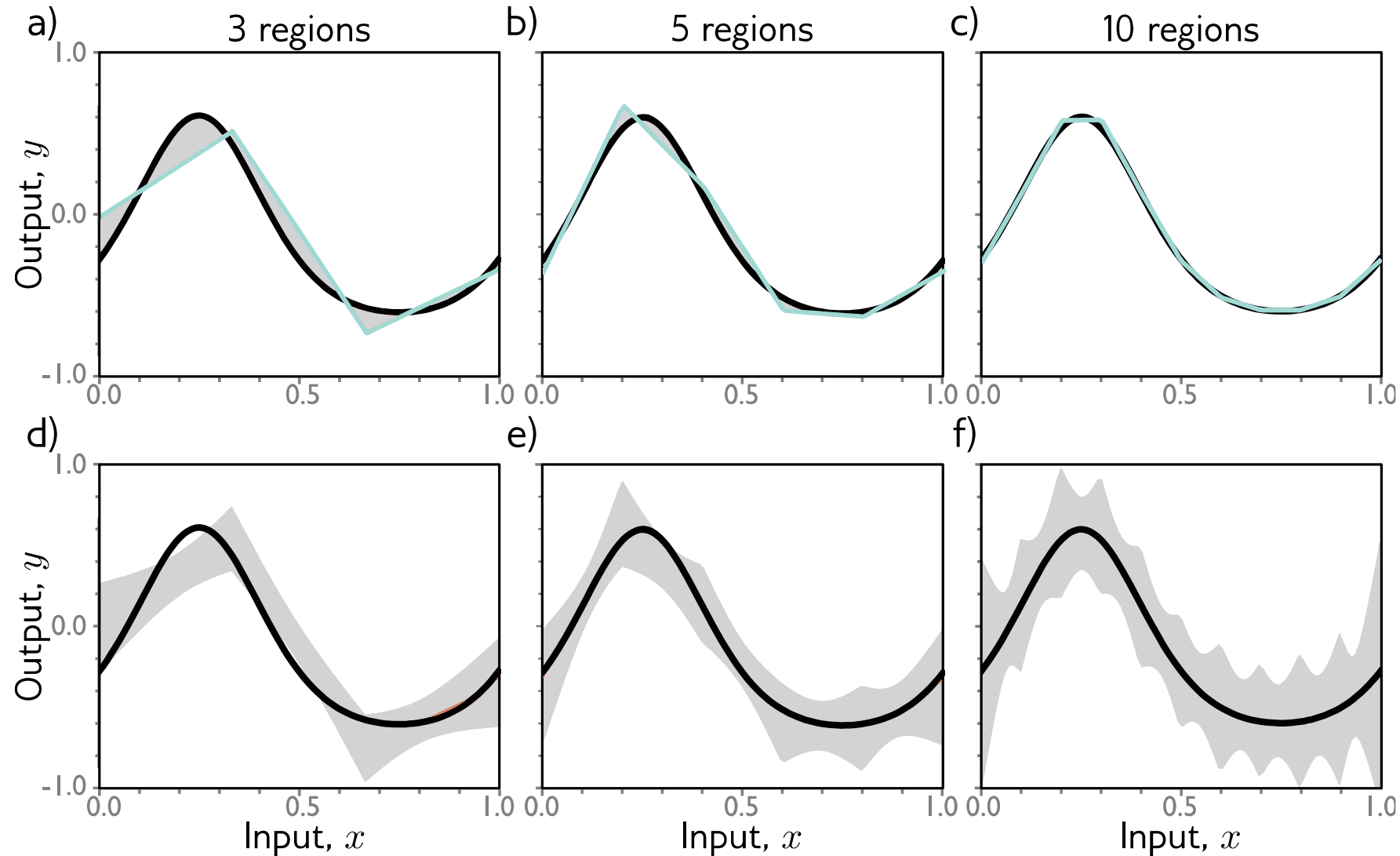
More training data



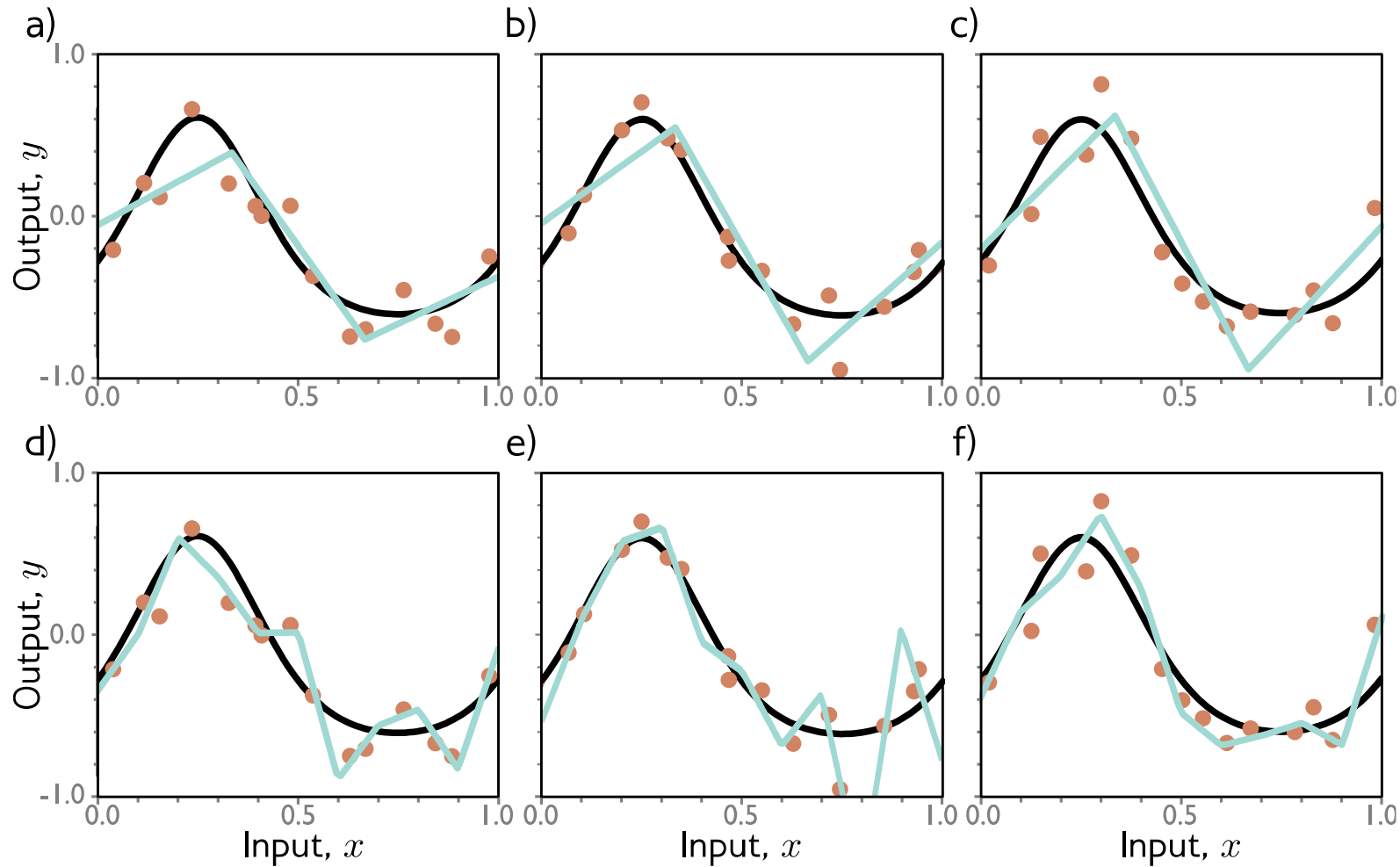
Reducing bias and bias-variance tradeoff



Reducing bias and bias-variance tradeoff

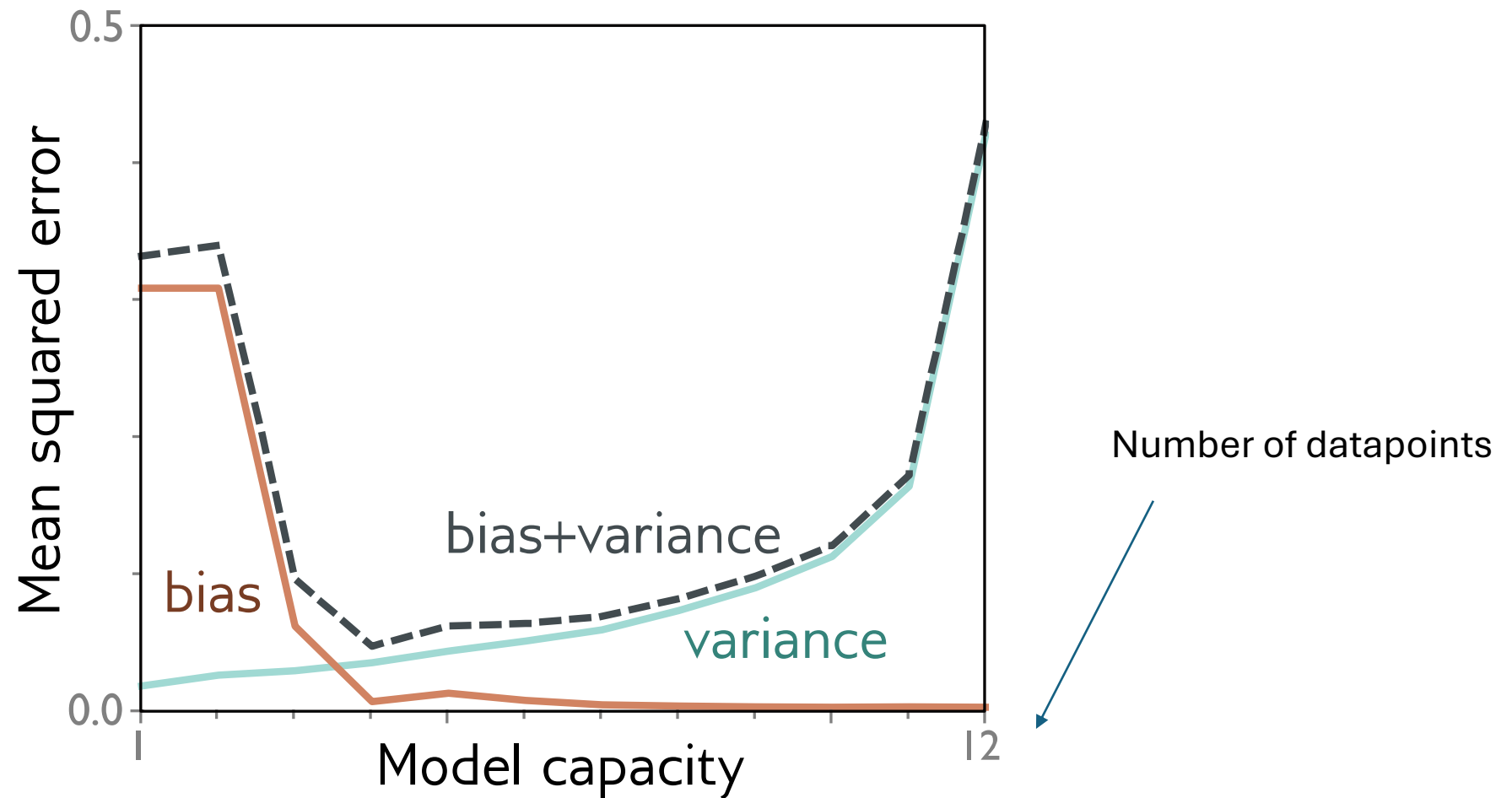


Why does variance increase? Overfitting



Describes the training data better, but not the true underlying function (black curve)

Bias and variance trade-off



Regularization

- Why is there a generalization gap between training and test data?
 - Overfitting (model describes statistical peculiarities)
 - Model unconstrained in areas where there are no training examples
- **Regularization** = methods to reduce the generalization gap
- Technically means adding terms to loss function
- But colloquially means any method (hack) to reduce gap

Explicit regularization

- Standard loss function:

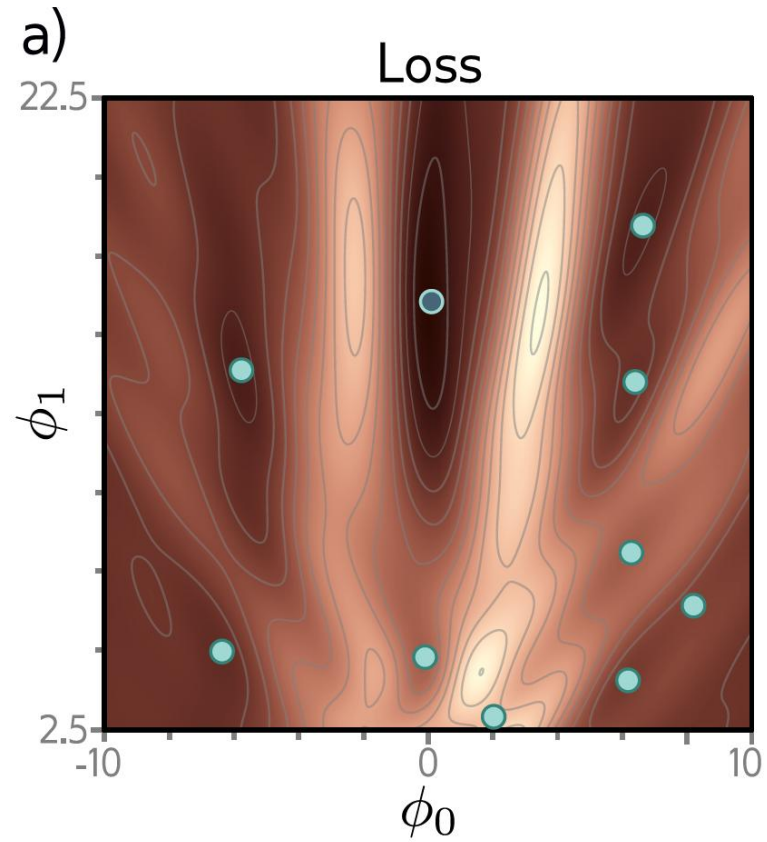
$$\begin{aligned}\hat{\phi} &= \operatorname{argmin}_{\phi} [L[\phi]] \\ &= \operatorname{argmin}_{\phi} \left[\sum_{i=1}^I \ell_i[\mathbf{x}_i, \mathbf{y}_i] \right]\end{aligned}$$

- Regularization adds an extra term

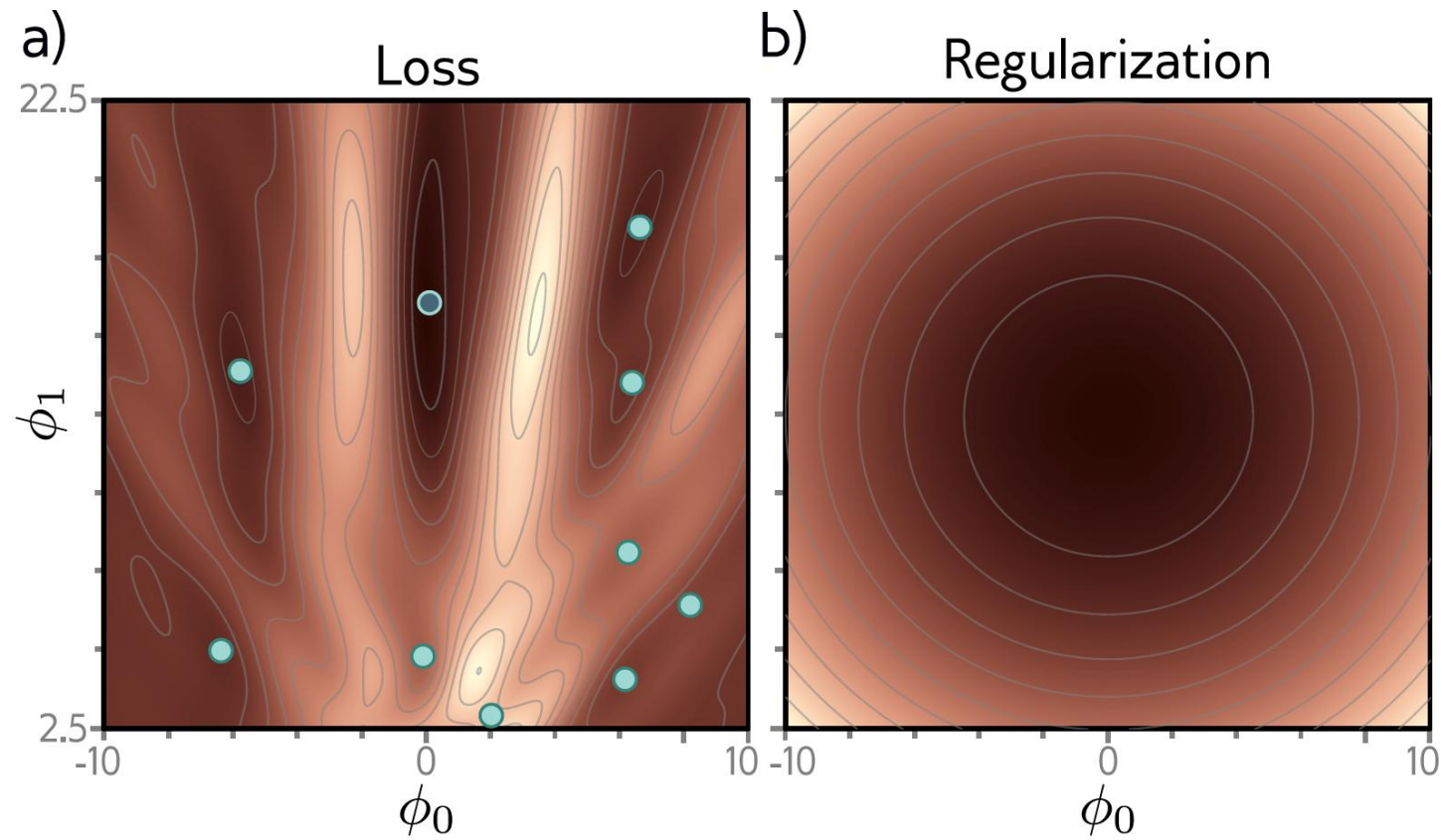
$$\hat{\phi} = \operatorname{argmin}_{\phi} \left[\sum_{i=1}^I \ell_i[\mathbf{x}_i, \mathbf{y}_i] + \lambda \cdot g[\phi] \right]$$

- Favors some parameters, disfavors others.
- $\lambda > 0$ controls the strength

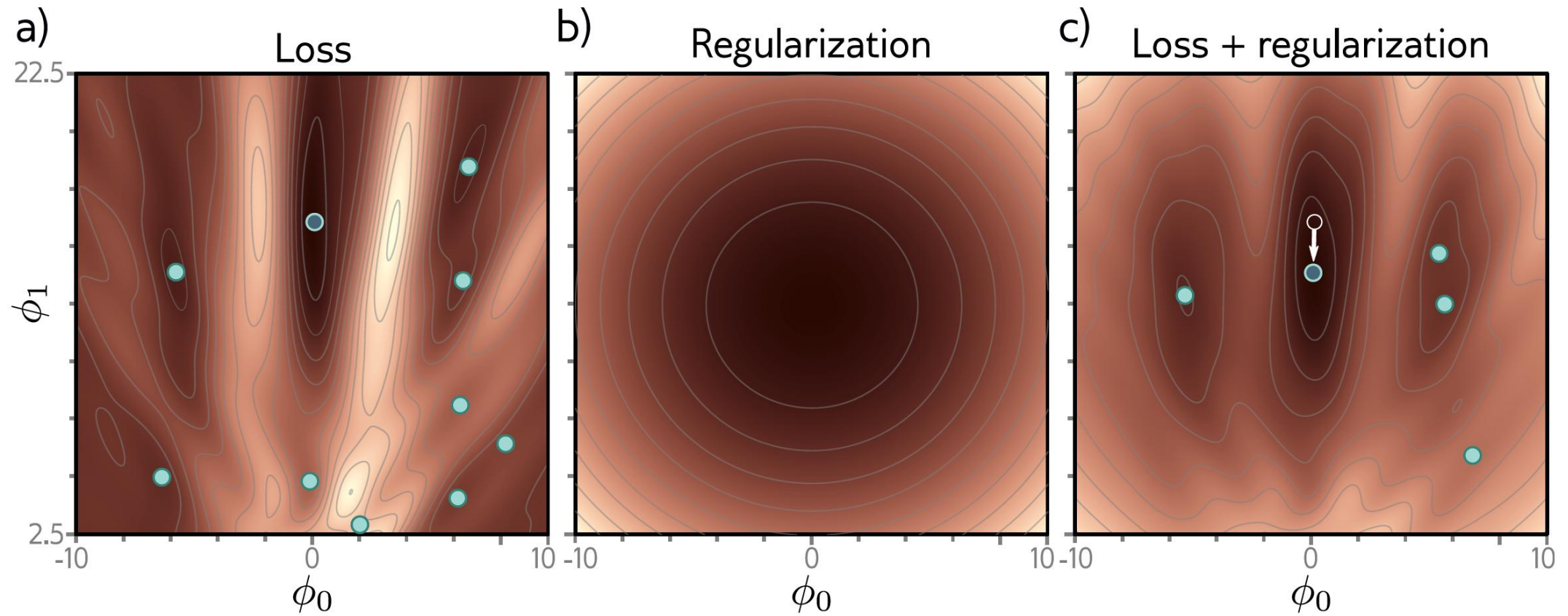
Explicit regularization – Gabor Example



Explicit regularization



Explicit regularization



L2 Regularization

- Can only use very general terms
- Most common is **L2 regularization**
- Favors smaller parameters

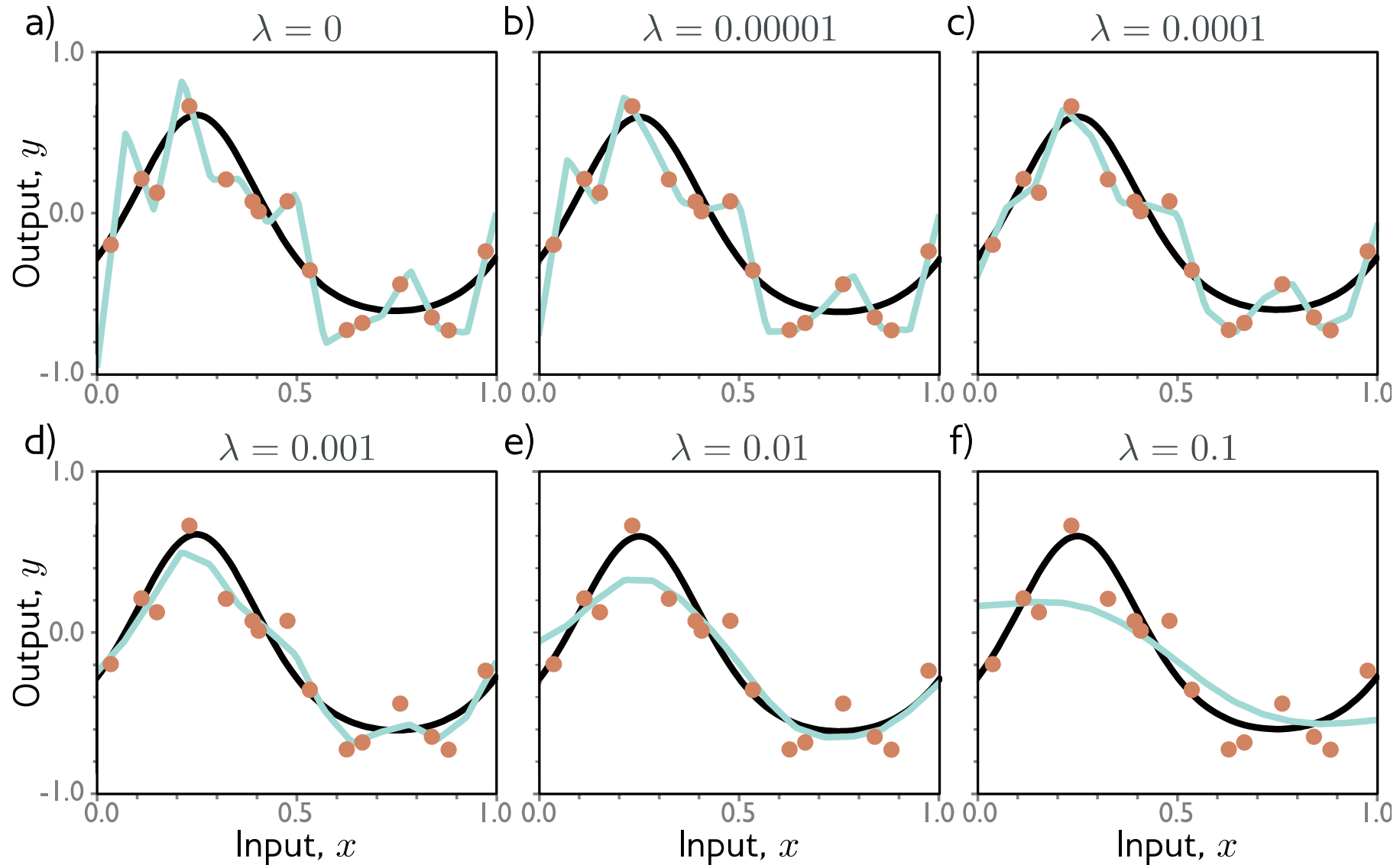
$$\hat{\phi} = \operatorname{argmin}_{\phi} \left[L[\phi, \{\mathbf{x}_i, \mathbf{y}_i\}] + \lambda \sum_j \phi_j^2 \right]$$

- Also called **Tikhonov regularization, ridge regression**
- In neural networks, usually just for weights and called **weight decay**

Why does L2 regularization help?

- Discourages slavish adherence to the data (overfitting)
- Encourages smoothness between datapoints

L2 regularization



L1 Regularization

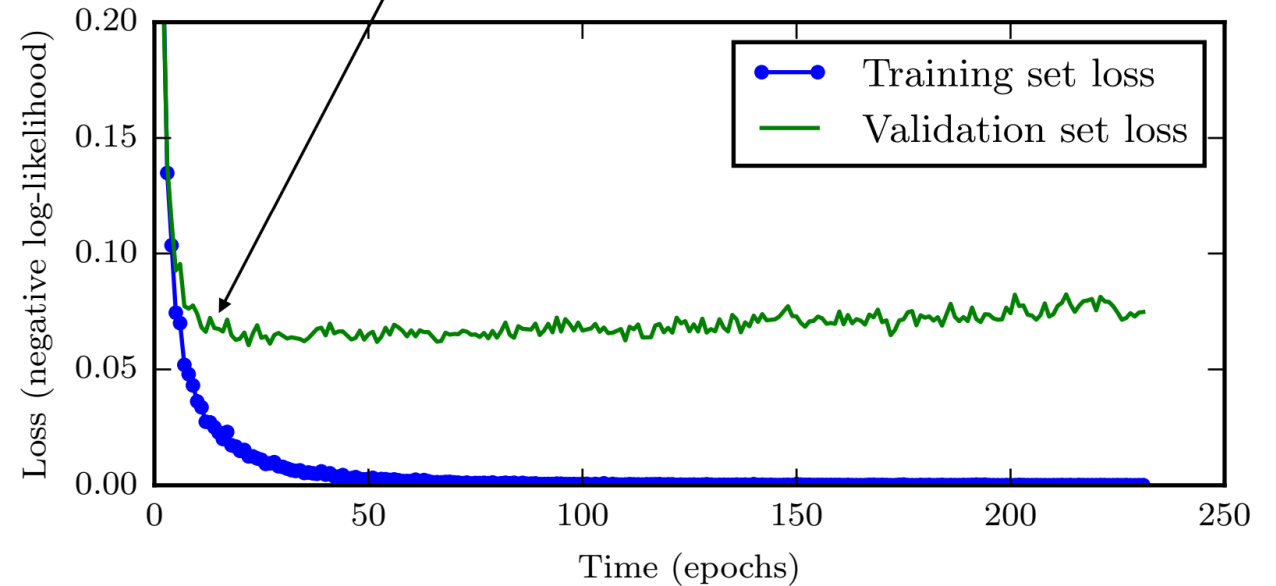
- Sometimes we want sparse models where most weight parameters should have an optimal value of zero
- L2 does not force parameters to be zero
- L1 regularization uses the sum of absolute values rather than the sum of squares.

$$\hat{\phi} = \operatorname{argmin}_{\phi} \left[L[\phi, \{\mathbf{x}_i, \mathbf{y}_i\}] + \lambda \sum_j |\phi_j| \right]$$

- This will encourage parameters to become zero, if λ is large enough, producing weights that are sparse (lots of zeros).
- This has been used as a feature selection mechanism (e.g. LASSO for linear models).

Early stopping as regularisation

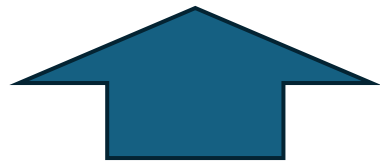
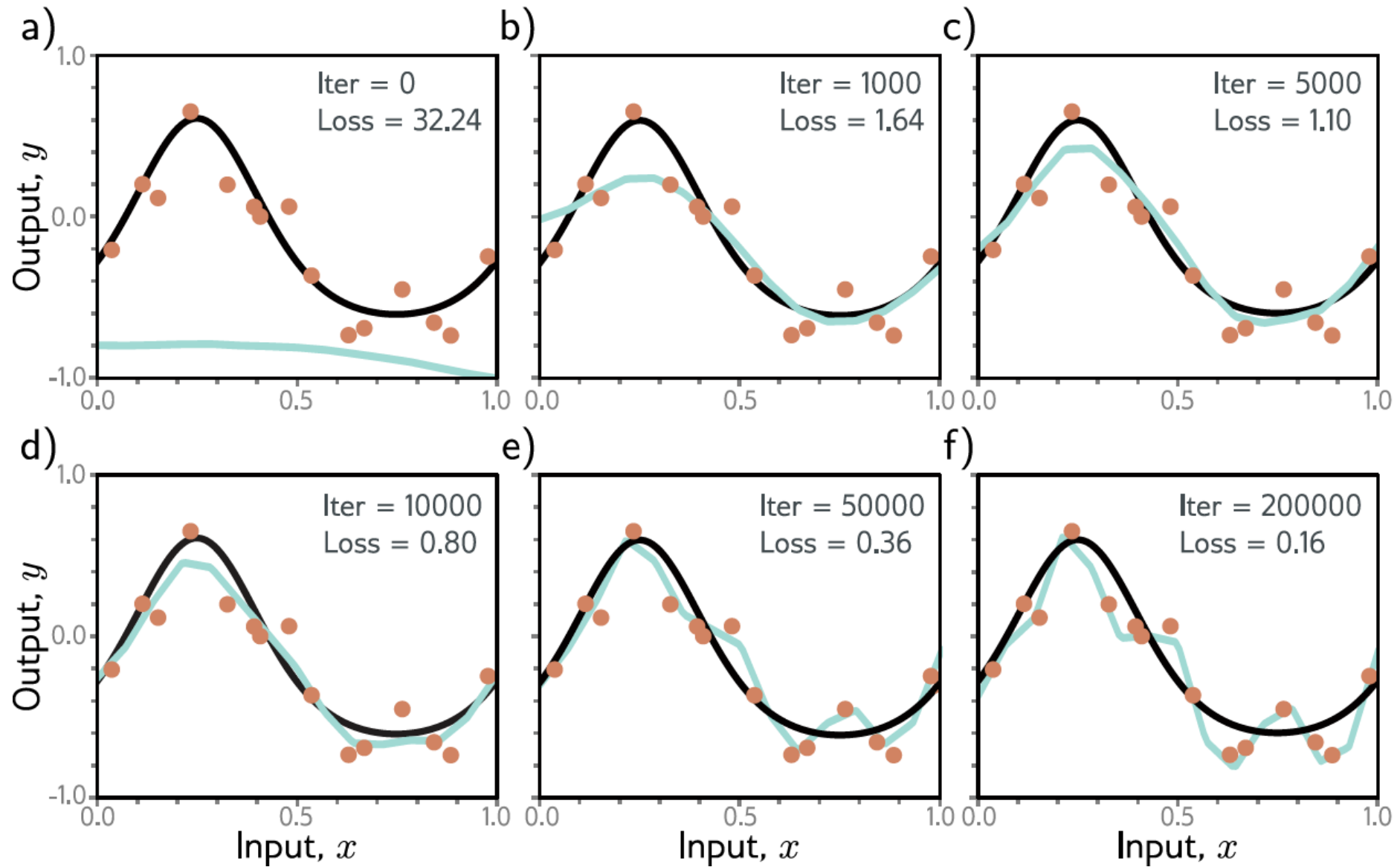
Early stopping: terminate while validation set performance is better



If we stop training early, weights don't have time to overfit to noise.

Weights start small, don't have time to get large.

Reduces effective model complexity.



Stop at 10,000 iterations before it starts to overfit.
(i.e. start modelling the noise in the data)

Using more training data improves generalization

Gathering more training data will improve the accuracy on unseen test data and so is a regularization approach.

More training data can be generated through data augmentation.



Affine
Distortion



Noise



Elastic
Deformation



Horizontal
flip



Random
Translation



Hue Shift



Easy with appropriate image transformations - although we have to be careful (not to turn a “b” into a “d” if we are training on letters)

Adversarial training

Adversarial examples that “fool networks” can be generated by adding in just the right type of noise.

Training on these adversarial examples (to predict the correct label) can make networks more robust and generalizable.

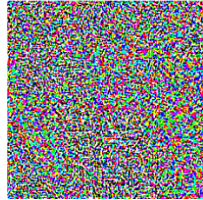
	$+ .007 \times$		$=$	
$\mathbf{x}$		$\text{sign}(\nabla_{\mathbf{x}} J(\boldsymbol{\theta}, \mathbf{x}, y))$		$\mathbf{x} + \epsilon \text{sign}(\nabla_{\mathbf{x}} J(\boldsymbol{\theta}, \mathbf{x}, y))$
$y = \text{“panda”}$		“nematode”		“gibbon”
w/ 57.7%		w/ 8.2%		w/ 99.3 %
confidence		confidence		confidence

Figure 7.8

Training on adversarial examples is mostly intended to improve security, but can sometimes provide generic regularization.

Another
approach to
regularization:

Ensembles Methods

Bagging (short for bootstrap aggregating) is a technique for reducing generalization error by **combining several models** (Breiman, 1994).

The idea is to train several different models separately, then have all the models vote on the output for test examples.

An example of a general strategy in machine learning called *model averaging*.

- Techniques using this strategy are known as *ensemble methods*.

Why does this work?

- Different models will usually not make all the same errors on the test set.

Ensemble Methods

Assume k regression models. each model makes an error ϵ_i on each example, variance $E(\epsilon_i^2) = v$, covariance $E(\epsilon_i \epsilon_j) = c$.

$$\begin{aligned}\mathbb{E} \left[\left(\frac{1}{k} \sum_i \epsilon_i \right)^2 \right] &= \frac{1}{k^2} \mathbb{E} \left[\sum_i \left(\epsilon_i^2 + \sum_{j \neq i} \epsilon_i \epsilon_j \right) \right], \\ &= \frac{1}{k} v + \frac{k-1}{k} c.\end{aligned}$$

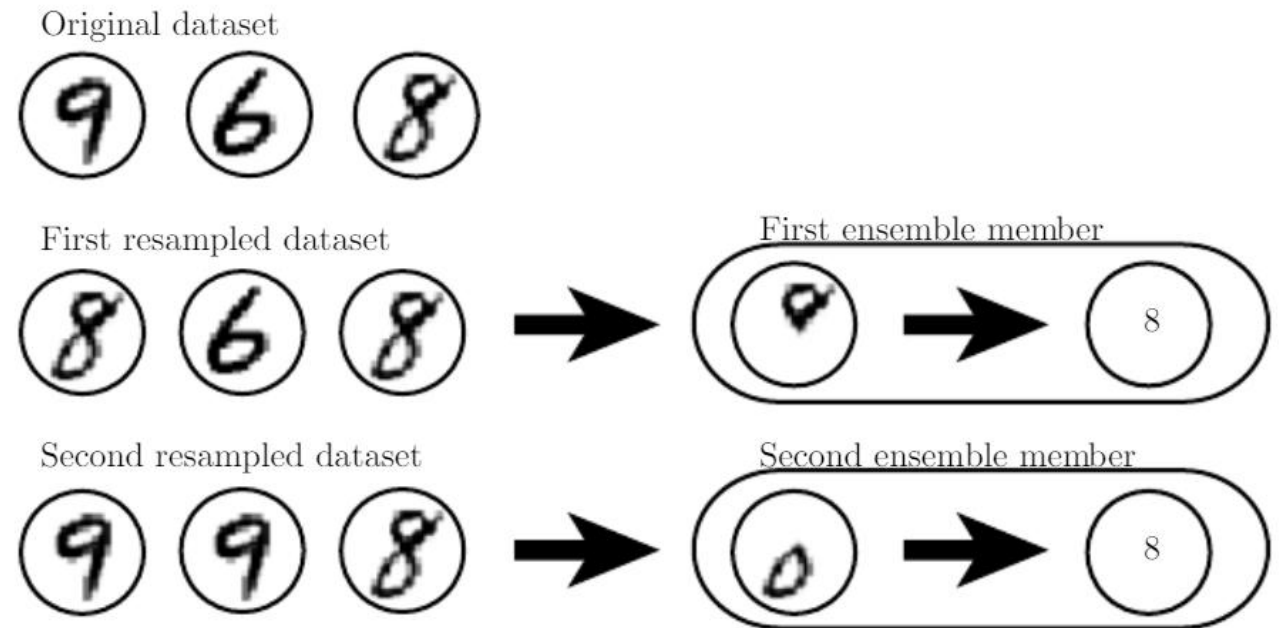
If errors are perfectly correlated ($c = v$) then model averaging does not help at all. However, for perfectly uncorrelated errors, $c = 0$, then the expected squared error is only $\frac{1}{k} v$, so the MSE of the ensemble would decrease linearly with ensemble size.

Bagging (aka 'Bootstrap aggregating')

Bagging uses the same model, training algorithm and objective function to be reused multiple times.

It constructs k different training sets by sampling from replacement from the original dataset. (missing some, with duplicates of others).

A similar approach where multiple neural networks are trained and they vote on the output is a very effective way of improving generalization. (Although computationally expensive.)

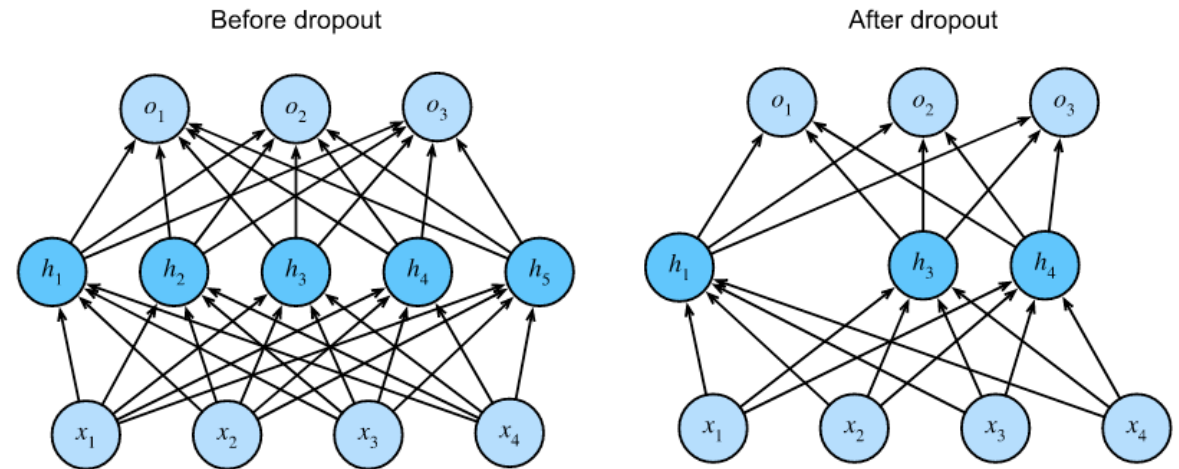


Dropout

At every step every neuron can have a probability p of being temporarily 'dropped out' for that training step (i.e. the output of these neurons is multiplied by zero).

p is the *dropout rate*.

After training there is no dropout but weights are scaled (say to 50%) so that neurons "on average" get the same input signals (or during training they are scaled up to give the same effect).



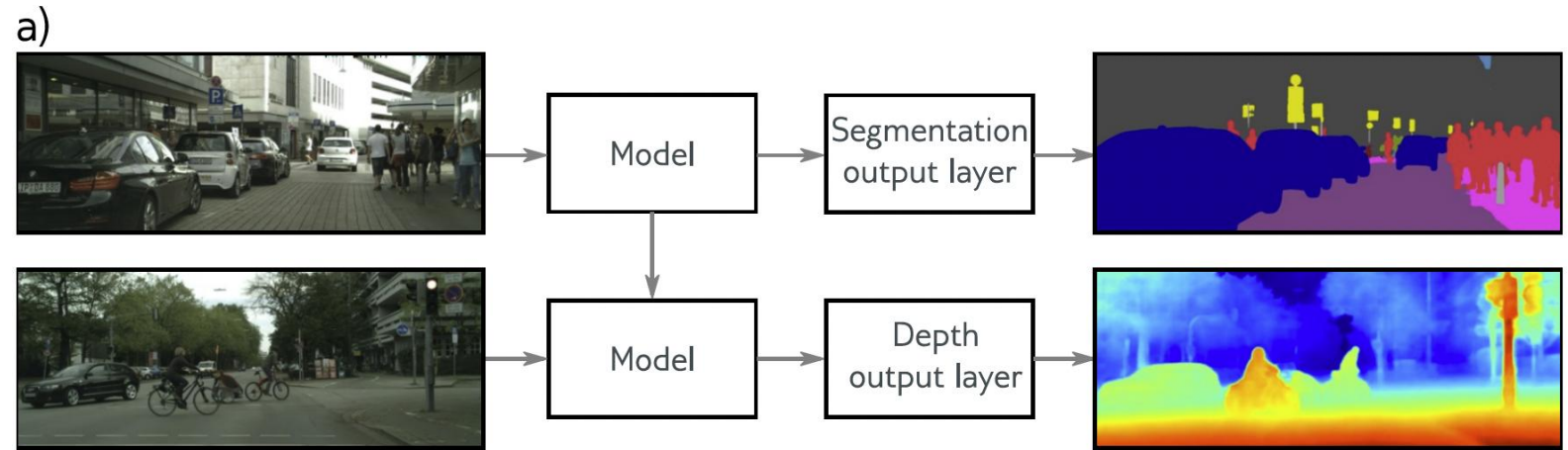
Dropout

Why does such a brutal technique work?

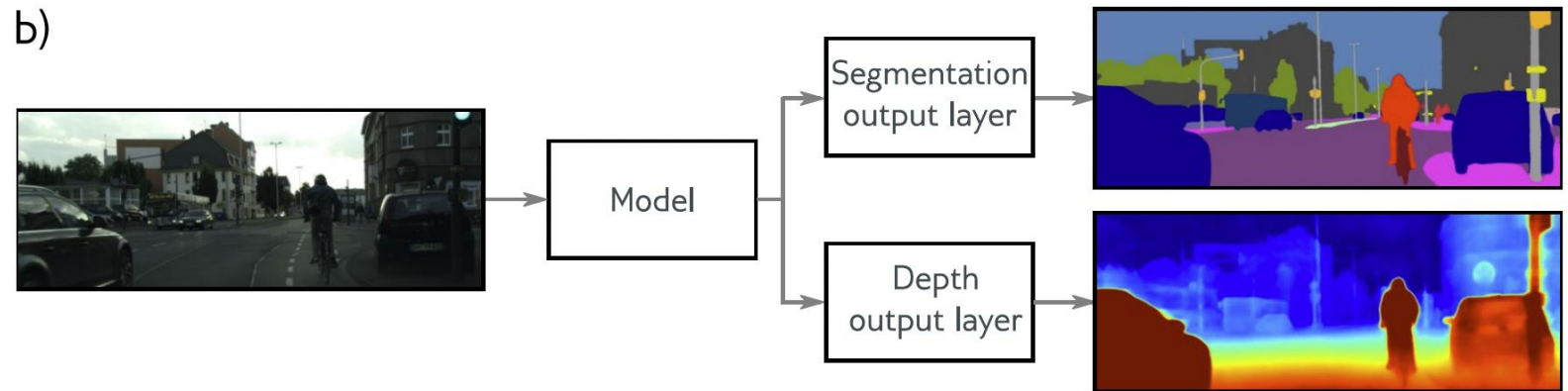
- Can be seen as an ensemble approach where one network (with shared weights) is being trained like lots of networks.
- Neurons cannot be overly dependent on each other, or on a small subset of inputs, so they become more independent and still work in different environments (like genes which get shuffled during reproduction but can still work).
- Forces redundancy (which can be wasteful, but can help make things more robust)

Model-level approaches for improving generalization

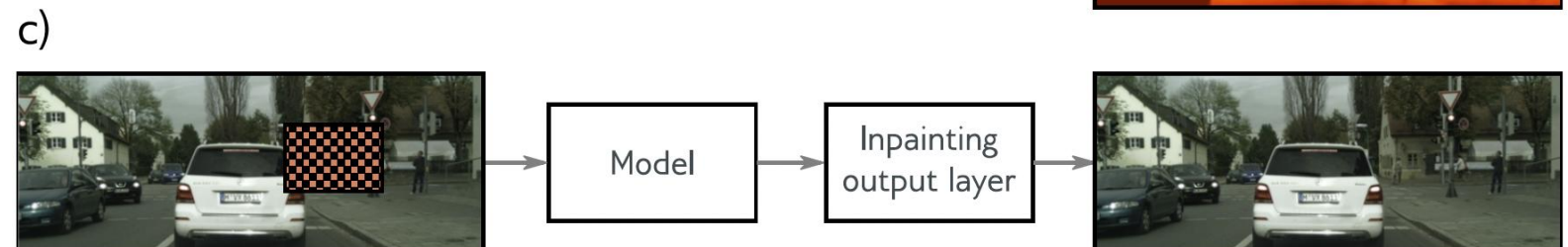
- Transfer learning



- Multi-task learning



- Self-supervised learning



Summary

- ❑ Regularization is any technique that helps minimize model overfitting to improve its generalization (i.e. improve its score on the validation or test set).
- ❑ Often you get better results by creating a high capacity deep neural network and regularize appropriately using:
 - ❑ Parameter Weight Norm Penalties like L1 and L2.
 - ❑ Early Stopping as a regularization method
 - ❑ Data Augmentation and adding noise to inputs to make the network more robust.
 - ❑ Ensemble methods (using multiple trained networks) and Dropout Regularization.
 - ❑ Transfer learning and multi-task learning.

Background Reading

Understanding Deep Learning (Simon Prince) Chapter 8 and 9.

[Chapter 7 of Goodfellow et al 2018](#)